

Client Scoping Note

The problem

Your on-call and platform teams are buried in log noise. Looking at a real sample of a similar size, about 44 percent of log lines are pure noise, health checks, cache warmups, debug flags, and most of the remaining lines are repeats of a small number of real incident patterns rather than unique events. Only a small number are things that actually need a person to act on. Right now someone has to sift through all of it, or worse, the noise causes real problems to get missed. This agent reads the stream, throws out what does not matter, works out what kind of problem each real issue is, and suggests a fix from an approved list, with a clear trail showing why it made that call.

What actually working in production means here

A demo that gets one example right in a chat window is not the same as a system that can handle real log volume. Three things break first in a quick and dirty version. Cost, since calling a big model on every single line, including the large share that is noise, gets expensive fast and does not hold up on a busy day. Trust, since a model that is free to make up its own fix will eventually suggest something nobody signed off on, which is a real risk during an incident. And silent misses, since a system that guesses instead of admitting it is not sure will sometimes wave through the wrong fix instead of asking a person first.

The trade off I am making

I am choosing to optimize for cost and accuracy over squeezing out every bit of speed. Grouping near identical log lines together before they ever reach the model, and only calling it once per distinct pattern instead of once per line, is what makes this affordable at scale. My build cuts model spend by about 97.7 percent compared to calling the model on every line, while keeping accuracy at or above target on the labeled evaluation set. The trade off is that the parts of this system that are pure rules, like filtering junk and grouping duplicates, do not need a model call, so an unusual log format the rules have not seen yet will fall through to the model instead of being caught early. That is the safer direction to fail in. I will also say plainly that my measured latency on the optimized build came in slightly above the target in the final run, which I explain in the results document, rather than leave out.

The biggest risks, and how I would handle them

1. A wrong fix gets applied automatically. I handle this by keeping every fix choice inside a fixed, approved list that gets checked twice, once when the agent generates it and once when my own script checks it afterward. The agent cannot write its own fix as free text, and anything it is not confident about goes to a person instead of being guessed.
2. A real incident gets mistaken for noise. I track how often this happens and keep the noise rules on the cautious side. When something is unclear, it gets flagged, not dropped.
3. Log formats change over time. A rule built around today's message patterns will quietly get worse as services change their logs. I would check on this regularly so any drift shows up before it causes a miss.

Rough scope and timeline for a two week engagement

Week one: build the pipeline that reads the logs, groups duplicates, filters out junk, sets up the model that classifies real incidents, and build the tool that measures accuracy, speed, and cost against real historical logs, running end to end.

Week two: add the part that sends unsure cases to a person, add retry handling for when something fails partway through, test it at ten times the normal daily volume, and hand off documentation plus a live walkthrough where I rerun the numbers in front of your team.